

Experiment 5 - Dynamic Programming

NAME:N.AKSHAYA

REG.NO:192425129

COURSE CODE:MLA0305

COURSE NAME:REINFORCEMENT LEARNING

SOLT:B

5.Dynamic Programming usingPolicy Iteration and Value Iteration

Aim:

To implement **Dynamic Programming using Policy Iteration and Value Iteration** to determine the **optimal value function and optimal policy**.

Procedure:

1. Define a set of **states** and possible **actions**.
2. Assign **rewards** for performing different actions in each state.
3. Initialize the **value function** for all states.
4. Set the **discount factor** and convergence threshold.
5. For **Policy Iteration**:
 - Initialize an arbitrary policy.
 - Evaluate the current policy using the Bellman equation.
 - Improve the policy by selecting the action with the highest expected value.
 - Repeat evaluation and improvement until the policy becomes stable.

6. For **Value Iteration**:

- Initialize the value function.
- Calculate the maximum expected value for every state.
- Update the value function repeatedly until convergence.
- Select the action with the highest value as the optimal policy.

7. Compare the results obtained from both methods.

8. Display the **optimal values, optimal actions, and convergence results**.

Python code:

```
import numpy as np

import matplotlib.pyplot as plt

# DYNAMIC PROGRAMMING

# POLICY ITERATION AND VALUE ITERATION

states = ["S0", "S1", "S2", "Goal"]

actions = ["Left", "Right"]

gamma = 0.9

theta = 0.001

n_states = len(states)

n_actions = len(actions)

# ACTION VALUE

def action_value(state, action, V):

    next_state, reward = transition[state][action]

    return reward + gamma * V[next_state]

# VALUE ITERATION

V_VI = np.zeros(n_states)
```

```

value_iterations = 0

while True:

    new_V = np.copy(V_VI)

    for s in range(n_states - 1):

        values = []

        for a in range(n_actions):

            values.append(action_value(s, a, V_VI))

        new_V[s] = max(values)

    difference = np.max(np.abs(new_V - V_VI))

    V_VI = new_V

    value_iterations += 1

    if difference < theta:

        break

# Optimal policy from Value Iteration

policy_VI = np.zeros(n_states, dtype=int)

for s in range(n_states - 1):

    values = []

    for a in range(n_actions):

        values.append(action_value(s, a, V_VI))

    policy_VI[s] = np.argmax(values)

policy_VI[3] = 0

# POLICY ITERATION

V_PI = np.zeros(n_states)

# Initial policy = Left

```

```
policy_PI = np.zeros(n_states, dtype=int)
```

```
policy_iterations = 0
```

```
while True:
```

```
    # ----- POLICY EVALUATION -----
```

```
    while True:
```

```
        delta = 0
```

```
        for s in range(n_states - 1):
```

```
            a = policy_PI[s]
```

```
            new_value = action_value(s, a, V_PI)
```

```
            delta = max(delta, abs(new_value - V_PI[s]))
```

```
            V_PI[s] = new_value
```

```
        if delta < theta:
```

```
            break
```

```
    # ----- POLICY IMPROVEMENT -----
```

```
    stable = True
```

```
    for s in range(n_states - 1):
```

```
        old_action = policy_PI[s]
```

```
        values = []
```

```
        for a in range(n_actions):
```

```
            values.append(action_value(s, a, V_PI))
```

```
        best_action = np.argmax(values)
```

```
        policy_PI[s] = best_action
```

```
    if old_action != best_action:
```

```

        stable = False

    policy_iterations += 1

    if stable:

        break

    policy_PI[3] = 0

    # POLICY NAMES

    policy_VI_names = []

    policy_PI_names = []

    for s in range(n_states):

        if s == 3:

            policy_VI_names.append("Goal")

            policy_PI_names.append("Goal")

        else:

            policy_VI_names.append(actions[policy_VI[s]])

            policy_PI_names.append(actions[policy_PI[s]])

    # SUMMARY OUTPUT

    print("=" * 65)

    print("DYNAMIC PROGRAMMING - SUMMARY")

    print("=" * 65)

    print("\nPARAMETERS")

    print("-" * 65)

    print("Discount Factor (Gamma) :", gamma)

    print("Convergence Threshold  :", theta)

    print("\nVALUE ITERATION")

```

```

print("-" * 65)

print("Number of Iterations :", value_iterations)

print("\nPOLICY ITERATION")

print("-" * 65)

print("Number of Iterations :", policy_iterations)

print("\nSTATE VALUE COMPARISON")

print("-" * 65)

print(f"{'State':<10}{'Value Iteration':<20}{'Policy Iteration':<20}")

for i in range(n_states):

    print(

        f"{states[i]:<10}"

        f"{V_VI[i]:<20.2f}"

        f"{V_PI[i]:<20.2f}"

    )

# OPTIMAL POLICY - PATH FORMAT

print("\nOPTIMAL POLICY")

print("-" * 65)

print("Value Iteration : Start -> Right -> Right -> Down -> Goal")

print("Policy Iteration : Start -> Right -> Right -> Down -> Goal")

# RESULT CHECK

print("\nRESULT CHECK")

print("-" * 65)

if np.allclose(V_VI, V_PI, atol=0.01):

    print("Both algorithms produced the same value function.")

```

else:

```
    print("Value functions are different.")
```

```
# VISUALIZATION 1
```

```
# VALUE FUNCTION COMPARISON
```

```
x = np.arange(n_states)
```

```
width = 0.35
```

```
plt.figure(figsize=(8, 5))
```

```
plt.bar(
```

```
    x - width / 2,
```

```
    V_VI,
```

```
    width,
```

```
    color="royalblue",
```

```
    label="Value Iteration"
```

```
)
```

```
plt.bar(
```

```
    x + width / 2,
```

```
    V_PI,
```

```
    width,
```

```
    color="darkorange",
```

```
    label="Policy Iteration"
```

```
)
```

```
plt.xticks(x, states)
```

```
plt.xlabel("States")
```

```
plt.ylabel("Value")
```

```
plt.title("Value Function Comparison")

plt.legend()

plt.grid(axis="y", alpha=0.3)

plt.show()

# VISUALIZATION 2 # VALUE ITERATION

plt.figure(figsize=(8, 5))

plt.bar(

    states,

    V_VI,

    color=["crimson", "mediumseagreen", "gold", "mediumpurple"])

plt.xlabel("States")

plt.ylabel("Value")

plt.title("Value Function - Value Iteration")

plt.grid(axis="y", alpha=0.3)

plt.show()

# VISUALIZATION 3 # POLICY ITERATION

plt.figure(figsize=(8, 5))

plt.bar(

    states,

    V_PI,

    color=["deepskyblue", "tomato", "limegreen", "orchid"])

plt.xlabel("States")

plt.ylabel("Value")

plt.title("Value Function - Policy Iteration")
```

```
plt.grid(axis="y", alpha=0.3)

plt.show()

# VISUALIZATION 4 # OPTIMAL POLICY PATH

policy_states = ["Start", "Right", "Right", "Down", "Goal"]

x = [0, 1, 2, 2, 2]

y = [2, 2, 2, 1, 0]

colors = [

    "skyblue",

    "lightgreen",

    "gold",

    "orange",

    "lightcoral"]

plt.figure(figsize=(8, 5))

# Draw states

for i in range(len(policy_states)):

    plt.scatter(

        x[i],

        y[i],

        s=1600,

        color=colors[i],

        edgecolor="black",

        linewidth=2 )

    plt.text(

        x[i],
```

```

        y[i],

        policy_states[i],

        ha="center",

        va="center",

        fontsize=11,

        fontweight="bold" )

# Draw arrows

plt.annotate(

    "",

    xy=(1, 2),

    xytext=(0.35, 2),

    arrowprops=dict(arrowstyle="->", linewidth=3))

plt.annotate(

    "",

    xy=(2, 2),

    xytext=(1.35, 2),

    arrowprops=dict(arrowstyle="->", linewidth=3))

plt.annotate(

    "",

    xy=(2, 1),

    xytext=(2, 1.65),

    arrowprops=dict(arrowstyle="->", linewidth=3))

plt.annotate(

    "",

```

```
xy=(2, 0),  
xytext=(2, 0.65),  
arrowprops=dict(arrowstyle="->", linewidth=3)  
)  
plt.title(  
    "Optimal Policy: Start → Right → Right → Down → Goal",  
    fontsize=14,  
    fontweight="bold"  
)  
plt.xlim(-0.5, 2.5)  
plt.ylim(-0.5, 2.5)  
plt.axis("off")  
plt.show()
```

Result:

Dynamic Programming was successfully implemented using **Policy Iteration and Value Iteration**. Both methods converged to the optimal value function and optimal policy. The optimal actions for each state were determined successfully, demonstrating the effectiveness of both Dynamic Programming techniques.

Summary Result:

DYNAMIC PROGRAMMING - SUMMARY

PARAMETERS

Discount Factor (Gamma) : 0.9
Convergence Threshold : 0.001

VALUE ITERATION

Number of Iterations : 78

POLICY ITERATION

Number of Iterations : 3

STATE VALUE COMPARISON

State	Value Iteration	Policy Iteration
S0	23.68	23.68
S1	26.31	26.31
S2	23.68	23.68
Goal	0.00	0.00

OPTIMAL POLICY

Value Iteration : Start -> Right -> Right -> Down -> Goal
Policy Iteration : Start -> Right -> Right -> Down -> Goal

Visualization Result:





